

TalentServ AI Hackathon — Group 11

Office Leave Agent Platform — Submission Pack

Repository: <https://github.com/devinrain258/talentserv-ai-hackathon-group-11-backend-db>

Date: May 2026

Executive Summary: ~\$20 Budget, Multi-Channel AI Platform

We delivered a **multi-channel office leave agent** while keeping runtime AI cost near **zero** and spending roughly **\$20** primarily on **agentic IDE credits** (Cursor), not on burning paid LLM API quotas.

Area	Deliverable	Typical cost
MCP server	7 tools, 2 resources, 1 prompt (FastMCP stdio)	\$0 runtime
AI (Grok)	Live advice on every MCP tool/resource + advise + WhatsApp advise via Puter	\$0 API (free Puter; x.ai optional)
Grok enrichment	Text header/footer + grok/grok_footer JSON on all tools; middleware safety net	Dev time only
SQLite HR backend	Employees, balances, apply/status/history, policy	\$0
Cursor IDE	MCP host + agentic development	~\$20 team credit
Graphify	Code knowledge graph, MCP server, Husky + GitHub Action	AST rebuilds \$0
Cache-cow hooks	Token-saving Cursor hooks	\$0
WhatsApp	Twilio Sandbox + FastAPI webhook + phone auth	Trial/sandbox \$0
Install UX	curl/PowerShell one-liners, uvx/pipx/venv fallback	\$0
Automated tests	37+ pytest cases (DB, Grok, WhatsApp)	\$0 CI

Judge pitch: Multi-channel leave agent—Cursor MCP with Grok on every tool, repo knowledge graph, token-optimized hooks, WhatsApp on Twilio—on Puter's free Grok path and ~\$20 in IDE credits.

1. Groomed Requirements Document

Problem

Employees need to check leave balances, submit requests, and get HR-style guidance without a custom mobile app or heavy HRIS integration.

Assumptions

- Demo scope: three employees (devin, nisha, gautam), local SQLite.
- Pending leave does not deduct balance until approved.
- MCP host is Cursor (stdio); compatible MCP clients may also work.
- Grok via Puter (free) or x.ai (paid); fallback rules if no API key.
- WhatsApp uses WHATSAPP_PHONE_MAP (E.164 to slug), not enterprise SSO.

In scope

- MCP tools: list/get employees, balances, apply leave, status, list requests, AI advise.
- MCP resources: employee directory, policy.
- MCP prompt: leave_assistant.
- Grok enrichment on all tool/resource responses (header, JSON bookends, footer).
- Graphify code graph, install scripts, CI sync.
- Cursor cache-cow hooks.
- WhatsApp (Twilio Sandbox): same DB, commands + advise with Grok.
- Clone-free MCP install (curl scripts).

Out of scope (v1)

- Production WhatsApp Business / Meta verification.
- Manager approval UI.
- MCP-over-HTTP from WhatsApp.
- Per-employee OAuth in MCP.

User stories

1. As an employee, I ask in Cursor for my leave balance to plan time off.
2. I submit annual/sick leave with dates and reason and see pending status.
3. I get AI recommendations after MCP actions and via advise_on_leave.
4. As a developer, I query the code graph instead of grepping entire files.
5. On WhatsApp, I text balance/apply on my registered phone.
6. As IT, I install MCP into any folder with one curl command.

Acceptance criteria

- All 7 MCP tools return JSON plus grok block (used_grok, source).
- apply_leave validates dates, type, balance; creates pending row.
- pytest passes for DB, enrichment, WhatsApp.
- .env loads from workspace (OFFICE_LEAVE_WORKSPACE).
- WhatsApp: registered phone gets help; unknown phone gets registration message.
- Graphify artifacts update on commit/CI per repo setup.

2. Solution Plan / Implementation Plan

Approach

Single Python monorepo: data layer (db.py) → MCP surface (server.py + mcp_enrichment.py) → channels (Cursor MCP, WhatsApp) → DX (install, graphify, hooks).

Implementation sequence

Phase	Deliverable	Modules
1	Schema + seed DB	src/sql/, init_db.py, db.py
2	MCP tools/resources	server.py
3	Grok + fallbacks	grok_client.py, models.py
4	Universal Grok wrapping	mcp_enrichment.py, middleware
5	Env loading	env.py
6	Graphify + CI	graphify-out/, .github/workflows/graphify.yml
7	Cache-cow hooks	.cursor/hooks/, docs/CACHE_COW.md
8	MCP install	scripts/mcp_install_lib.py
9	WhatsApp	src/whatsapp/*, docs/WHATSAPP_DEMO.md
10	Tests + docs	tests/, README.md

Responsibilities (fill team names)

Member	Focus
--------	-------

TBD	DB, MCP tools
TBD	Grok / Puter, enrichment
TBD	WhatsApp / Twilio
TBD	Graphify, CI, install
TBD	Tests, submission

3. Product / Technical Architecture

Clients

- **Cursor IDE** — MCP host (stdio).
- **WhatsApp** — Twilio Sandbox → HTTP webhook.
- **No separate web frontend.**

Backend

- **FastMCP** (src/server.py): 7 tools, 2 resources, 1 prompt.
- **FastAPI** (src/whatsapp/twilio_webhook.py): POST /whatsapp/webhook, TwiML replies.
- **Shared logic:** src/db.py, src/models.py.

Database

- SQLite (data/employees.db): employees, leave_balances, leave_requests.
- Init: office-leave-init-db.

Authentication

Surface	Mechanism
MCP (demo)	Employee slug in tool args
WhatsApp	WHATSAPP_PHONE_MAP; self-only commands
Twilio webhook	X-Twilio-Signature validation

Integrations

- Puter OpenAI-compatible API (Grok, default).
- x.ai (optional).
- Twilio WhatsApp Sandbox.

- Graphify (knowledge graph MCP).
- Cursor MCP host.

AI usage

- **Runtime:** Grok for advise_on_leave, per-tool recommendations, WhatsApp advise.
- **Development:** Cursor Agent for design, implementation, testing, merge, debugging.
- **Graphify:** AST-only updates (no LLM cost on routine graphify update).

Key design decisions

1. db.py is single source of truth; WhatsApp does not spawn MCP stdio.
2. WhatsApp gets short text; MCP gets full grok bookends.
3. Fallback rules when API keys absent.
4. GrokEnrichmentMiddleware prevents skipping enrichment on MCP tools.

4. Test Plan and Test Cases

Strategy

- Automated: pytest on temp SQLite (DATABASE_PATH per test).
- Manual: Cursor MCP calls; WhatsApp sandbox help → balance → apply → status.
- CI: no live Twilio/Grok; mocks for Grok HTTP.

Test suites (~37 cases)

Suite	Focus
test_tools.py	CRUD leave, validation, filters (8)
test_grok_enrichment.py	Fallback, mock Grok, all tools header/footer, middleware (12+)
test_whatsapp_commands.py	Phone map, parser, handlers (12)

Sample cases

ID	Scenario	Expected
T01	get_leave_balance(nisha)	success; annual=10, sick=4
T02	apply_leave valid	status=pending
T03	insufficient balance	error mentions Insufficient

T04	unknown employee	error lists slugs
T05	no API keys	used_grok=false, fallback-rules
T06	MCP enrichment	Grok advisory header; grok_footer in JSON
T07	WhatsApp balance nisha	ParseError (self-only)
T08	unregistered phone	Not registered message
T09	WhatsApp apply	Submitted request in reply

Run tests

```
pytest tests/ -q
```

5. Detailed Critical Review

Strengths

- Clear separation: db vs MCP vs WhatsApp.
- ToolResponse pattern throughout.
- Puter-first Grok with fallbacks.
- Broad pytest coverage.
- Strong DX: curl install, graphify, cache-cow, docs.

Security

- Do not commit .env (Twilio, Puter tokens).
- MCP demo: slug in args, no OAuth.
- WhatsApp: config-based phone map.
- Twilio signature validation (TWILIO_SKIP_SIGNATURE dev-only).

Limitations

- No manager approval workflow.
- WhatsApp: command parser; full NLU only on advise.
- Local/ephemeral tunnels for WhatsApp demo.

Technical debt

- Optional shared tool executor for future MCP client from WhatsApp.
 - Production WhatsApp needs Meta verification and persistent host.
-

6. Agentic Coding Evidence

Phase	AI tools used
Requirements	Scoped MCP + Grok on all tools; WhatsApp plan
Design	db vs MCP vs Twilio; Puter vs x.ai
Implementation	mcp_enrichment, grok_client, whatsapp package
Testing	test_grok_enrichment, test_whatsapp_commands
Review	master merge; GrokSuggestions restore
Debugging	Twilio inbound URL; Grok visibility MCP vs WhatsApp

Human: Twilio account, .env, phone map, run uvicorn + tunnel.

7. Source Code and Deployment

Clone and MCP

```
git clone https://github.com/devinraina258/talentserv-ai-hackathon-group-11-backend-db.git
cd talentserv-ai-hackathon-group-11-backend-db pip install -e ".[dev,graphify]" copy
.env.example .env office-leave-init-db pytest python -m src.server
```

WhatsApp

```
pip install -e ".[whatsapp]" office-leave-whatsapp # cloudflared tunnel --url
http://127.0.0.1:8000 # Twilio: https://<tunnel>/whatsapp/webhook (POST)
```

Environment variables

- DATABASE_PATH, PUTER_AUTH_TOKEN, GROK_PROVIDER, GROK_MODEL
- TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, WHATSAPP_PHONE_MAP
- OFFICE_LEAVE_WORKSPACE (MCP)

Documentation

- README.md, docs/MCP_SETUP.md, docs/WHATSAPP_DEMO.md, docs/CACHE_COW.md

MCP tools

list_employees, get_employee, get_leave_balance, apply_leave, check_leave_status, list_leave_requests, advise_on_leave

License

MIT

End of submission pack — Group 11